Shiksha — Project Report & Delivery Plan

Prepared for: the Shiksha engineering team **Scope:** current state of the platform, what it can do today, what remains, and a work-distribution plan for a team of ~6.

---

1. Executive summary

Shiksha (`shikshacom.com`) is a **feature-rich online education platform** built as **one Django backend + four React front-ends** (public site, student app, teacher app, admin panel) sharing a single sign-on across subdomains. It delivers **two products on one account system**: *Academy* (Class 8–12 board education) and *Skill Dev* (a guest- expert marketplace), plus shared social and live-class features.

Maturity in one line: the product is a **broad, working MVP** — almost every end-user journey is implemented end-to-end (auth, enrollment, live classes, coursework, chat, forum, payments, the skill marketplace, an admin control plane). What it is *not* yet is **hardened**: automated tests, CI/CD, observability, a shared front-end package, and several pieces of dead/duplicated code are the main gaps, alongside a clear list of **new features** the team wants (SMS, richer notifications, new tabs/apps).

This report inventories what's done, states what the platform can do, lists what's left, and proposes **six workstreams** that map 1:1 onto the accompanying interactive **Team Work Board**.

---

2. What's been built (feature inventory)

Legend: ● **Done & working** · ● **Works, needs hardening / polish** · ● **Not started**

2.1 Identity, accounts & access

Capability	Status	Notes
Email/password signup with verification	● **Done**	UUID users, email is the login, 24h verify tokens, Resend email.
Single-password, multi-identity accounts	● **Done**	One account → up to 5 learner profiles + a teacher identity + admin.
"Who's watching" profile picker + PINs	● **Done**	Dependent profiles, 4–6 digit switch PINs.
Context model (account / learner / teacher) in JWT	● **Done**	Switch identities without re-login.
Cross-subdomain SSO via cookies	● **Done**	`.shikshacom.com` httpOnly cookies, auto-refresh on 401.
Roles (Student / Teacher / Admin / Guest)	● **Done**	Multi-role accounts supported.
Teacher two-track model (Academy + Skill)	● **Done**	With the asymmetric "Guest→Faculty allowed, Faculty→Guest blocked" rule.

Capability	Status	Notes
Password reset (code → ticket → set)	● Done	No emailed links; 6-digit codes.
Per-profile content access gate	● Polish	Correct design (`has_active_subscription` scoped to learner profile); needs a full audit that every content endpoint uses it.

2.2 Academy (Class 8–12)

Capability	Status	Notes
Catalog: Board → Course → Subject → Chapter	● Done	Prices in paise, time-boxed subscriptions.
Enrollment requests + admin approval (manual UPI)	● Done	UTR + receipt upload, admin review.
Free enrollment + Razorpay paths	● Polish	Razorpay webhook present; needs idempotency/reconciliation hardening.
Live classes (LiveKit)	● Done	Create/join/pause/end, attendance, in-room chat, raise hand, teacher controls.
Session recordings (Bunny CDN)	● Done	Signed direct upload, per-student watch progress.
Assignments (create/submit/grade/ download-all)	● Done	Teacher CRUD + file attachments + bulk download.
Quizzes (author/take/attempts/review)	● Done	Questions, choices, attempts, graded answers.
Study materials (per subject/chapter)	● Done	Upload with file validation.
Chapter coverage / course progress	● Done	Teacher-ticked shared state, progress bars.
Private (1:1) & group video sessions	● Done	Requests, scheduling, reschedule, instant meetings, join-by-code, per-session chat.

2.3 Skill Dev (expert marketplace)

Capability	Status	Notes
Expert directory + categories	● Done	Public browse, expert profile pages.
Teacher screening pipeline	● Done	Application → interview slot → interview → evaluation → tier.
1:1 skill sessions (book/confirm/join/review)	● Done	LiveKit room, post-session reviews.
Self-paced skill courses	● Done	Course → section → lecture → enrollment → progress.
Expert ad subscriptions (promotion)	● Done	Admin-approved paid placement.
Skill payments (free/manual/gateway)	● Polish	Parallel to Academy payments; same hardening needs.

2.4 Social, comms & content

Capability	Status	Notes
Direct & course chat (WebSocket)	● Done	Conversations, course rooms, blocking, directory.
Forum (threads, comments, upvotes)	● Done	Tags, notifications.
Notification / activity feed + bell	● Polish	In-app live bell over WebSocket; **no** push notifications or email digests yet; WS host fallback is hard-coded.

Capability	Status	Notes
Public content (blogs, current affairs, counselling, placements)	● Done	GNews-backed current affairs; i18n only on the public site.

2.5 Admin control plane

Capability	Status	Notes
Users & teacher approvals	● Done	List/detail, approve teachers, manage roles.
Course / board management	● Done	CRUD for catalog.
Enrollment requests & enrollment management	● Done	Approve/revoke/reactivate, batch roster.
Payments & live payment-mode settings	● Done	Toggle free/manual/Razorpay live (no restart) via a settings singleton.
Skill approvals, experts, sessions, courses, ad subs	● Done	Full skill-side moderation.
Forum moderation	● Done	Thread/comment management.

2.6 Platform & infrastructure

Capability	Status	Notes
Async jobs (Celery + Beat)	● Done	Session reminders, auto-complete sessions, expire subscriptions.
WebSockets (Channels + Redis)	● Done	Notifications, chat, live sessions, course presence.
Env-aware settings (dev/prod)	● Done	Dispatcher, per-env hosts/origins/SSL.
Tests	● To-do	`tests.py` files are largely stubs.
CI/CD, linting, formatting	● To-do	Not present in the repo.
Observability (logging/metrics/alerts)	● To-do	Console logging only; no Sentry/metrics.
Shared front-end package	● To-do	`AuthContext`, `apiClient`, `config/urls.js` are hand-duplicated across 3 apps.
Docs / `.env.example` / `package.json` in FE repos	● Polish	Developer Guide now exists; env templates & Vite scaffolding still missing.

---

3. What the platform can do today (by user)

A student can: sign up and verify; create child/dependent profiles with PINs; browse the catalog; enroll (free, manual-UPI with receipt, or Razorpay); attend live classes; watch recordings with resume; submit assignments; take quizzes and see results; read study material; track course progress; book private/group video sessions; browse and book guest experts; take skill courses; chat directly and in course rooms; use the forum; and receive live in-app notifications.

A teacher can: apply via the Faculty or Guest track; once approved, run two dashboards (Academy + Expert); manage assigned classes; create assignments, quizzes, materials, and recordings; host live classes and private/group sessions; mark chapter coverage; view students and submissions; and (as a guest expert) list a profile, set availability, take bookings, sell skill courses, and buy promotion.

An admin can: manage users and roles; approve teachers and screen experts; manage the course catalog; review and approve enrollment requests; revoke/reactivate access; switch the platform's payment mode live; moderate the forum; and oversee all skill-side activity (experts, sessions, courses, ad subscriptions, orders).

---

4. Current-state assessment

Strengths

- **Genuinely complete feature surface** — the hard, integration-heavy parts (LiveKit video, Bunny uploads, cross-subdomain SSO, the multi-identity context model, a pluggable live-switchable payment system, a full admin plane) are all working.
- **Sound core architecture** — clean Django app boundaries, UUID keys, service layers, a single source of truth for URLs, and a well-considered auth/identity design.
- **Operational flexibility** — payment mode and free-trial are admin-toggleable without a deploy.

Risks / gaps

- **No automated test safety net** and **no CI** — every change is a manual-QA gamble.
- **Dead & duplicated code** in the backend (nested app copies, a committed virtualenv, stray backups) inflates the repo and misleads new developers.
- **Front-end duplication** — three apps carry hand-copied auth/client/url files; a bug fixed in one can silently persist in the others.
- **No observability** — failures are invisible until a user reports them.
- **Design inconsistency** — four apps grown independently; no shared design system.
- **Notifications are in-app only** — no SMS, push, or email digests yet.

---

5. What's left to do

Grouped so it maps directly onto the team workstreams in §6 and the work board.

5.1 Bugs & technical debt (cleanup)

- Remove **nested duplicate app folders** under `enrollments/` (`accounts/`, `assignments/`, `quizzes/`, `livestream/`, `skills/`) — dead code, not loaded by Django.
- Remove the **committed** `forum/venv/` virtualenv and add it to `.gitignore`.
- Delete stray `*.py.bak` / `*.save.1` editor backups and `enrollments/skills/Pasted Image.png`.
- Remove/relocate the root `z` status dump and confirm-or-remove legacy `gmail_auth.py` (email runs through Resend).
- **Audit the access gate**: ensure *every* Academy content endpoint uses `has_active_subscription(learner_profile=get_active_profile(request))` — never the account alone. (Highest-value correctness task.)
- Fix the **hard-coded WebSocket host fallback** in `useNotificationSocket.js` (`api.shikshacom.com`) — read it from `config/urls.js`.
- Reconcile the `/api` **placement inconsistency** in `config/urls.js` across apps.
- Resolve **naming drift** ("three apps" in comments vs four deployed).

5.2 Engineering foundations (currently missing)

- **Automated tests**: start with the auth/context flow, the subscription access gate, and the payment-mode branching; then per-app API tests.
- **CI/CD**: lint + test on PR, build + deploy pipelines per app.
- **Linting/formatting**: `ruff` / `black` (backend), `eslint` / `prettier` (front-ends).
- **Shared front-end package**: extract `AuthContext`, `apiClient`, `config/urls.js`, `ProfileSwitcher`, guards into one internal package (or a monorepo workspace).
- **Repo hygiene**: commit `package.json` + Vite config per front-end, add `.env.example` files (backend + each app), and a top-level README pointing at the Developer Guide.
- **Observability**: structured logging, error tracking (e.g. Sentry), uptime/latency metrics, and payment/webhook audit logging.

5.3 New features requested

- **SMS system** — a provider integration (e.g. MSG91 / Twilio) for OTP login/verify, enrollment-approval alerts, and session reminders. Cleanest as a **new Django app** (`sms`) exposing a small send API + templates, wired into the existing event points (signup, enrollment approval, session reminder Celery tasks).
- **Notification system upgrade** — add **web push** (service worker + VAPID), **email digests**, and an **in-app notification preferences** screen; unify Academy/Forum/ Skill notifications behind one service so any new feature can emit a notification.
- **New WebSocket-powered tabs** — the pattern is established (consumer + `routing.py` + `asgi.py` + a front-end hook + a new route/tab). Candidate live tabs: a presence/ "who's online" panel, live quiz/poll during classes, real-time submission grading feed for teachers.
- **New Django app(s)** — for any net-new domain (e.g. `sms`, `notifications` service, `analytics`/ reporting, `search`). Follow the established app shape (models → serializers → services → views → urls, plus consumers/tasks as needed).
- **Design / UX** — a shared **design system** (tokens, components), a UI consistency pass across the four apps, accessibility (WCAG) and i18n beyond the public site.

5.4 Recommended roadmap (phased)

Phase	Theme	Headline items
Phase 1 — Stabilise	Make changes safe	Cleanup dead code; access-gate audit; first tests + CI; observability; `.env.example` + repo hygiene.
Phase 2 — Communicate	Reach users	SMS app; notification upgrade (push + email digests + preferences); shared FE package.
Phase 3 — Extend & scale	New value	New WebSocket tabs; analytics/reporting; search; payment hardening; design-system rollout; PWA/mobile.

---

6. Work distribution — six workstreams for ~6 people

Each workstream has a clear owner, a scope, and example tasks. These map **1:1** to the swim-lanes/tags in the **Team Work Board** (the interactive HTML file shipped alongside this report), so progress is tracked there.

> Roles can flex: one person can own a workstream and pull help from others during peaks. The board lets you reassign any task to anyone.

WS-1 · Backend Core & New Apps *(owner: Backend Lead)*

Scope: new Django apps, models/migrations, service logic, the **SMS app**, the unified **notification service**. **Example tasks:** scaffold `sms` app + provider client; OTP send/verify endpoints; wire SMS into signup/enrollment/reminders; build a `notifications` service other apps emit to; data-model changes for new features.

WS-2 · Realtime & WebSockets *(owner: Realtime Engineer)*

Scope: Channels consumers, ASGI routing, the front-end socket hooks, and **new live tabs**. **Example tasks:** fix the hard-coded WS host; add a presence/"who's online" consumer + tab; live in-class polls/quiz; teacher live-submission feed; reconnection/auth hardening of the notification socket.

WS-3 · Frontend Features (Student & Teacher) *(owner: Product FE Engineer)*

Scope: new pages/tabs and feature work inside `src_student_` and `src_teacher`. **Example tasks:** notification-preferences screen; new WebSocket-backed tabs UI; expert/ academy UX improvements; wiring new backend endpoints into pages and nav.

WS-4 · Frontend Platform & Design *(owner: Design/Platform FE Engineer)*

Scope: the **shared front-end package**, the **design system**, cross-app UI consistency, accessibility, i18n. **Example tasks:** extract `AuthContext` / `apiClient` / `config/urls.js` into one package; build design tokens + a component library; restyle the four apps to one system; accessibility audit; extend i18n beyond the public site.

WS-5 · Admin, Payments & Integrations *(owner: Integrations Engineer)*

Scope: the admin panel, **payment hardening**, and third-party integrations (SMS/email providers, Razorpay, Bunny, LiveKit). **Example tasks:** Razorpay webhook idempotency + reconciliation; payment/webhook audit logging; admin reporting screens; provider-key management; email-digest sender.

WS-6 · QA, DevOps & Tech Debt *(owner: QA/DevOps Engineer)*

Scope: tests, **CI/CD**, repo cleanup, security, observability, docs. **Example tasks:** test the auth/context flow + access gate + payment branching; set up CI (lint+test on PR) and deploy pipelines; remove dead/duplicated code & the committed venv; add `.env.example` + README; integrate Sentry + uptime metrics; security pass (rate limits, headers, secrets).

Suggested allocation (6 people)

Person	Primary workstream	Secondary (peak help)
Dev 1	WS-1 Backend Core & New Apps	WS-5
Dev 2	WS-2 Realtime & WebSockets	WS-3
Dev 3	WS-3 Frontend Features	WS-4
Dev 4	WS-4 Frontend Platform & Design	WS-3
Dev 5	WS-5 Admin, Payments & Integrations	WS-1
Dev 6	WS-6 QA, DevOps & Tech Debt	any

7. How to run the work with the Team Work Board

The companion file `Shiksha_Team_Work_Board.html` is a self-contained tracker (open it in any browser). It is **pre-loaded with the backlog from §5** and set up for the six workstreams above.

What you can do in it

- **Kanban board** — move every task across **Backlog → To Do → In Progress → Review → Done** (drag-and-drop or the move buttons).
- **Assign work** — set the **assignee** (your 6 people, editable) and **workstream/ category** on any task; add new tasks with the **+ Add task** button.
- **Track progress** — the **Dashboard** view shows overall completion, status breakdown, per-person workload, and per-category counts. The **By Person** view shows each developer's queue and a personal progress bar.
- **Filter** — by person, category, or workstream to focus a stand-up.
- **Never lose data** — changes persist automatically; use **Export** to download a JSON snapshot (and **Import** to restore or move it to another machine).

Suggested cadence: assign the §5 cleanup + foundations tasks to WS-6 and the access-gate audit to WS-1 first (Phase 1), run a short daily stand-up filtered by person, and use the Dashboard's completion ring as the weekly status number.

Companion deliverables: `Shiksha_Developer_Guide.pdf` (how the system works) and `Shiksha_Team_Work_Board.html` (the live tracker this report references).